

Lightweight confidential and traceable multi-stakeholder data processing

Steylor Barros¹[?????], Eduardo Falcão²[0000-zzz-zzz-zzzz], and Andrey Brito¹[0000-0002-7350-8599]

¹ Universidade Federal de Campina Grande, Campina Grande/PB, Brazil

² Universidade Federal do Rio Grande do Norte, Natal/RN, Brazil

steylor.barros@ccc.ufcg.edu.br

eduardo@dca.ufrn.br

andrey@computacao.ufcg.edu.br

<https://www.lsd.ufcg.edu.br/>

Abstract. We leverage technologies such as confidential computing and distributed ledgers to propose an architecture for multi-stakeholder, confidential data processing. Our approach aims to protect non-interactive data-processing applications in scenarios where multiple parties (e.g., data controllers, application developers, and infrastructure managers) distrust one another. We generalize previous results, describe the architecture and requirements of the involved components, and present an implementation focused on lightweight execution in the AWS cloud. Our implementation focuses on Python applications, showing overheads as low as $X\%$.

Keywords: Device Attestation, User Authentication, Zero Trust Security, Virtual Private Network (VPN), Remote Attestation, SPIFFE, SPIRE, Multi-Factor Authentication (MFA)

1 Introduction

Confidential data marketplaces must simultaneously support asset exchange, access control, and third-party computation over sensitive inputs. That combination creates a difficult systems problem. A marketplace that accepts externally supplied applications cannot treat code execution as an isolated implementation detail, because the same platform also handles persistent metadata, payment and access logic, storage backends, and the datasets that motivate the marketplace in the first place. In practice, the problem is not merely how to “run code safely,” but how to prevent build-time and run-time compromise from expanding into a full platform compromise.

The most direct implementation path would be a centralized confidential virtual machine [1] that hosts the marketplace-facing logic, local artifact storage, and the execution orchestrator in the same trust domain. That design is operationally attractive because it minimizes deployment complexity and avoids

inter-service coordination. However, it also creates the wrong security shape for the problem: the same node that validates access and manages persistent sensitive state also becomes the place where untrusted or semi-trusted workloads are prepared and executed.

This paper argues that such a concentration of privilege is fundamentally ill-suited for high-assurance marketplaces. It is proposed an architecture designed under the assumption that component compromise is an eventual certainty rather than a remote possibility. By decomposing the marketplace into distinct trust domains—control, build, and execution [5]—the system ensures that a failure or malicious exploit in one phase cannot propagate to the entire application. Our focus is on minimizing the “blast radius” of errors and attacks, ensuring that even if the build or execution planes are corrupted, the platform’s core sensitive assets and persistent state remain protected.

2 Background

2.1 Confidential computing

Confidential Computing [1] is a paradigm that focuses on protecting data while it is being processed. It leverages hardware-level memory encryption to make a process or an entire Virtual Machine inaccessible to the host hypervisor. By using an ASID (Address Space Identifier), the processor applies cryptography to a memory context; the ciphertext is decrypted transparently by a key stored directly in the memory controller when the VM that owns that context accesses the data. Confidential Virtual Machines are recognized by a special bit, called the C-bit, which instructs the CPU to restrict access to the corresponding memory pages. Any attempt by the hypervisor to map those pages into its own address space raises a processor exception, enforcing hardware-backed isolation even against a privileged adversary.

2.2 Blockchain

A blockchain [6] is a peer-to-peer distributed system that records transactions with strong integrity guarantees. Transactions are grouped into blocks; each block contains a cryptographic hash of the previous block, creating a chain in which every entry implicitly attests to the entire preceding history. Altering any block invalidates all subsequent hashes, so corrupting the ledger requires recomputing more than fifty percent of the combined processing capacity of all participating peers—an effort generally considered computationally infeasible in practice. Blockchains also support Smart Contracts [4], which are programs deployed and executed directly on the network. A Smart Contract can register and govern access to relevant data, such as the encryption keys and access permissions used in our system. The main drawback of blockchains is their inherent transaction cost: every write operation requires consensus among the majority of peers before a block is accepted, which introduces both financial cost and latency.

2.3 IPFS

The InterPlanetary File System (IPFS) [3] addresses the challenge of storing large data volumes that do not integrate well with a blockchain due to the costs described above. IPFS is a structured distributed system that uses a Distributed Hash Table (DHT) to locate files across the network. Files are identified by their content hash—a deterministic, one-way function that produces a unique identifier for a given sequence of bytes; any modification to the content produces a different hash. The DHT ensures that data remains accessible to all peers by managing replication and routing efficiently across the network, making IPFS a practical complement to blockchain-based access control for large binary artifacts.

3 Problem definition and solution requirements

The core challenge addressed by the Dataset and Application Marketplace (DNAT) involves a transaction between two mutually distrusting parties: a *data buyer*, who owns a confidential application, and a *dataset provider*, who holds sensitive data. The objective is to enable a computation in which the buyer’s application processes the provider’s dataset without exposing either asset to the other party or to the underlying cloud infrastructure.

A straightforward implementation would deploy a single Confidential Virtual Machine (CVM) responsible for all platform functions: receiving uploads, managing the IPFS storage layer, interacting with the Smart Contract, resolving and installing Python dependencies, and running the application against the dataset. While operationally simple, this monolithic design concentrates all privileges within one trust domain. The critical vulnerability emerges during dependency resolution: installing external packages to satisfy an application’s requirements necessarily executes arbitrary third-party code with the same privileges as the marketplace layer, the IPFS node, and the blockchain client. A maliciously crafted package—or a compromised dependency in the supply chain—could exploit this phase to exfiltrate stored datasets, tamper with marketplace metadata, or establish a persistent backdoor affecting future executions. A build-time compromise thus propagates immediately into a full platform compromise, making dependency resolution a disproportionately dangerous entry point.

This observation reframes the core problem. Secure execution of the application at runtime is necessary but not sufficient. An equally important concern is preventing a compromise during the build and dependency resolution phase from reaching the broader platform. The problem must therefore be treated as a *privilege separation problem*: distinct operational phases—control, build, and execution—carry fundamentally different risk profiles and must occupy separate trust domains with different network-access policies, persistence rights, and failure blast radii.

Three design consequences follow from this reframing. First, the component managing marketplace state, blockchain interaction, and IPFS operations must

never directly execute dependency installation logic for user-supplied applications. Second, the build phase, where external network access is unavoidable, must be isolated from both persistent sensitive storage and the execution runtime. Third, the execution phase, during which the sensitive dataset is consumed, must operate without any outbound network interface, eliminating the primary exfiltration channel even under a complete runtime compromise.

A secure solution must satisfy the following requirements:

4 Proposal

4.1 Architecture

The DNAT platform is organized around three Confidential Virtual Machines, each assigned a distinct operational role and a distinct set of persistence rights. **CVM1** acts as the control plane, **CVM2** as the isolated build plane, and **CVM3** as the isolated execution plane. The central design principle is privilege separation: a compromise in one plane must not grant access to the assets of the others. (...)

4.2 Implementation

The architecture described above is realized using Firecracker microVMs [2] as the primary isolation primitive for both the build and execution environments. The current prototype instantiates each CVM as a containerized service for development purposes; in a production deployment, each service would run inside a hardware-backed Confidential VM [1]. (...)

5 Implementation

This section describes how the three planes are realized in the prototype, so that the security properties evaluated later can be traced to concrete mechanisms rather than to design intent alone. The system is deployed as three containers on a shared runtime—**dnat-client** (CVM1), **dnat-builder** (CVM2), and **dnat-executor** (CVM3)—and is contrasted with two baselines: a single-container deployment that colocates the same assets (A) and a no-isolation variant that removes the microVM entirely (M0). Each plane is a container in the current prototype, standing in for a hardware-backed confidential VM [1] in a production deployment; because the planes interact only through their declared interfaces and isolation boundaries, this substitution changes the execution substrate but not the architecture or the security argument evaluated in Section 6. We therefore do not report confidential-computing overheads, and the analysis that follows holds under a fully confidential deployment.

5.1 Control Plane (CVM1)

The control plane hosts the marketplace API, a local IPFS node, and the blockchain client. Application and dataset registration, access checks, and execution orchestration are exposed through a small HTTP API. On application registration, CVM1 does not install dependencies itself: it assembles a build bundle (the source file plus a manifest of declared dependencies) and forwards it to the builder. When the builder returns an `application.ext4` artifact, CVM1 encrypts it before publication. The encryption uses AES-256-GCM over a gzip-compressed artifact, with a versioned binary envelope (magic header, length prefix, JSON header carrying the IV, authentication tag, and metadata). Only the resulting encrypted blob and a manifest are published to IPFS; the plaintext artifact never leaves the control plane. Access control is enforced on chain: an execution request first calls `hasAccessByIds`, and the workflow aborts with *access denied* if the caller has not purchased access to both the dataset and the application. A purchased right is recorded under the key `keccak256(encryptedDatasetHash, encryptedApplicationHash, buyer)`, bound to the encrypted IPFS CIDs of the two assets and the buyer’s address rather than to mutable asset identifiers, so the grant is stable under off-chain re-indexing and is verified before any bundle is assembled.

5.2 Build Plane (CVM2)

The builder exposes a single `/build` endpoint that accepts a bundle and launches an ephemeral Firecracker microVM to resolve dependencies. Unlike the executor, the build microVM is given a network interface, because resolving and compiling Python wheels may require egress to package indexes. Egress is constrained on the host side: a TAP device is created and iptables rules `REJECT` traffic from the guest toward private ranges (10.0.0.0/8, 172.16.0.0/12, 192.168.0.0/16, 127.0.0.0/8, 169.254.0.0/16, and the host TAP address) while allowing the remaining outbound traffic via masquerading. Inside the guest, an artifact builder produces a read-only `ext4` image containing the application source under `/app`, its `site-packages`, a lock file of resolved dependencies, and a marker file used later for disk discovery. Dependencies are first wheel-built and then installed with `-no-index` from the local wheelhouse, and reusable wheels are persisted in a size-pruned cache that is the builder’s only durable state. After the build, the host returns only the artifact and build metadata; the per-request build state is discarded.

5.3 Execution Plane (CVM3)

The executor exposes a single `/execute` endpoint. For each request it copies a fresh root overlay, creates new input and output `ext4` disks, optionally attaches the application artifact disk read-only, and launches a Firecracker microVM with one vCPU and 512 MiB of memory and—crucially—no network interface.

The guest does not receive disk roles out of band; it discovers them by scanning candidate block devices for marker files, then extracts the workspace and runs `workspace/run.sh`. The launcher script tries a small, fixed set of invocation strategies so that applications can accept the dataset either as `-dataset data/dataset.csv` or as a positional argument; standard output and error are captured and persisted to the output disk. Synchronization between host and guest uses the serial console: the guest writes an `EXECUTION_COMPLETE` marker, after which the host issues a graceful shutdown, escalating to termination if needed. The host then mounts the output disk and derives the HTTP response solely from the persisted result. A `cleanup` trap unmounts every loop device and removes the entire per-run working directory on exit, which is what makes execution state ephemeral. Figure 1 summarizes this per-request lifecycle.

```
execute(bundle):                                # one /execute request [host]
    workdir  = fresh per-run directory
    rootfs   = copy-on-write overlay of the base image
    in_disk  = new ext4, read-only, holding the bundle    # INPUT role
    out_disk = new ext4, writable, tagged by a marker    # OUTPUT role
    vm = Firecracker(rootfs, [in_disk, out_disk, app_disk?],
                      vcpu=1, mem=512 MiB, NO network)    # no eth0
    vm.start()
    wait_serial(vm, "EXECUTION_COMPLETE")    # guest signals completion
    vm.shutdown(graceful, then kill on timeout)
    result = read(out_disk)                    # {rc, stdout, stderr}
    on exit: umount loop devices; rm -rf workdir    # ephemeral

guest (disk roles are discovered, not passed in):
    find disks by marker; extract the bundle into /tmp/exec
    run workspace/run.sh    (--dataset PATH, else positional)
    write {rc, stdout, stderr} to OUTPUT; print EXECUTION_COMPLETE
```

Fig. 1. Per-request lifecycle of the execution plane (CVM3): a fresh, networkless Firecracker microVM per run, disk roles discovered by marker rather than passed in out of band, completion signaled over the serial console, and a cleanup trap that leaves no residual state.

5.4 Application Contract and Dependency Reuse

An application is registered as unmodified source together with a manifest that declares its third-party dependencies. The builder (CVM2) resolves those dependencies and bundles them, with the source, into the read-only `application.ext4` artifact, and the executor (CVM3) runs the application through the fixed `workspace/run.sh` entry point, passing the dataset by a small set of conventions (a `-dataset` flag or a positional path). Because dependencies are shipped as ordinary `site-packages`

inside the artifact—rather than linked into a trusted-execution enclave at build time—an application that follows this simple contract runs with arbitrary declared dependencies, and runs unchanged across all three environments, without recompilation or enclave-specific adaptation. This is the reuse property that Section 6 contrasts with the enclave-based original DNAT, where third-party dependencies require recompilation or adaptation.

5.5 Asset Integrity and Baselines

Assets are bound to the chain by content hash: registration records the SHA-256 of the artifact, which lets a consumer detect tampering between IPFS and the contract. The first baseline (`docker/baseline-vm.compose.yaml`, environment A) runs the identical control, build, and execution logic, including Firecracker isolation for the build and execution microVMs, but inside a single container whose filesystem holds the IPFS repository, the wheel cache, and the executions store, and whose environment holds the asset-encryption key and the wallet private key. The only structural difference from the compartmentalized deployment is therefore the colocation of these assets, which is exactly the variable the security evaluation isolates.

A second baseline removes isolation altogether. With `DNAT_NO_MICROVM` set, the executor runs `vm/run-direct.sh`, which executes the identical `workspace/run.sh` contract directly in the container’s namespace—no Firecracker, no per-run input and output disks, and no network removal—while emitting the same `{returncode, stdout, stderr}` result so the request path is otherwise unchanged. This no-isolation variant (M0) is the lower endpoint of the spectrum in Section 6: the application inherits the container’s network, environment (including the control-plane secrets), and persistent filesystem, which lets the evaluation separate the microVM’s operational cost from the containment it provides.

6 Evaluation

The architectural argument of this paper is comparative: the compartmentalized design should be more robust against compromises than less-separated alternatives while still offering an acceptable operational cost. We evaluate the implementation along two axes. For *security*, we organize the analysis with the STRIDE model [7] and show, category by category, how the design neutralizes—or explicitly fails to neutralize—each threat class, mapping every claim to a concrete test. For *cost*, we measure the latency, build time, and resource price of the isolation strategy.

The comparison spans up to four points on an isolation spectrum, summarized in Table 2: a no-isolation monolith (M0), a centralized deployment that adds ephemeral microVMs but colocates all assets (Environment A), the compartmentalized design that also separates the planes (Environment B), and the SGX-based original DNAT [5]. Because A and B share the same Firecracker microVMs, any cost or security difference between them isolates the effect of

compartmentalization; M0 isolates the cost and the security delta of the *microVM* primitive itself; and the SGX reference frames the reuse and dependency limitations that motivate our changes in technology and design.

6.1 Methodology

We exercise the running prototype with a small suite of purpose-built applications, each designed to attack one STRIDE category, over a synthetic, PII-free dataset. Tests that target the execution plane submit an application bundle directly to the executor, reproducing the same execution contract the control plane generates, so that the variable under study — the execution microVM — is isolated from unrelated services; tests that target lateral movement inspect the executor environment, and the access-control test drives the marketplace API. Each adverse application emits machine-readable evidence markers that a harness collects, and each test is mapped to the STRIDE category it supports (Table 1), so that every observation connects to a property rather than merely listing successful checks.

Environments A and B were both measured live and end to end. The centralized baseline (Environment A) keeps execution under the same Firecracker isolation as the compartmentalized design (Environment B), so the only independent variable between the two is the separation of planes; any measured difference therefore reflects compartmentalization, not a weaker execution sandbox. The no-isolation baseline (M0)—which runs the application directly in the control namespace, without launching a microVM—was measured on the same host, and the on-chain gas cost was measured on Hardhat; together these complete the comparative matrix (Table 2), leaving only the SGX reference column, which refers to the original DNAT work.

6.2 Security Evaluation with STRIDE

Table 1 maps each STRIDE category to the mechanism that addresses it and the test that provides evidence. We discuss each in turn.

Spoofing. Execution is gated by the smart contract. After registering the synthetic dataset on chain, an execution request issued without a prior `purchaseAccess` call is rejected with *access denied* by the `hasAccessByIds` check (T5), before any execution bundle is prepared. The control plane therefore refuses to move data or code toward the executor for an unauthorized caller.

Tampering. Assets are bound to the chain by content hash: the on-chain content hash matches the file’s SHA-256, so a consumer can detect tampering between storage and use (T5). The application artifact and the wheel cache are mounted read-only; a build-time attempt to write to the cache fails with `Permission denied` (T2b), so cached state cannot be silently poisoned by a build.

Repudiation. Registration and access purchase are recorded on the distributed ledger (T5), making the provenance of a dataset and the fact of an access grant auditable rather than deniable.

Table 1. Security evaluation organized with the STRIDE model. For each threat category we give the mechanism in the layered design and the test that provides evidence. “Contained” means the adverse behavior stayed within the plane that ran it.

STRIDE category	Mechanism in the design	Test	Observed outcome (measured)
Spoofing	On-chain access gate (purchaseAccess/hasAccessByIds) checked before any bundle is prepared	T5	Unauthorized run rejected with <i>access denied</i> at the control plane, before any data or code moves toward the executor.
Tampering	SHA-256 content hash on chain; application fact and wheel cache mounted read-only	T5, T2b	On-chain content hash equals the file SHA-256; a build-time write to the cache is denied (Permission denied , read-only mount).
Repudiation	Distributed ledger records registration and access purchase	T5	Registration and access actions are recorded on chain and auditable.
Information disclosure	Execution microVM launched with no network interface; no control-plane secrets in the plane; sensitive stores in a separate namespace	T1, T1b, T4	7/7 outbound channels blocked (no eth0); no secrets in the environment; IPFS repository, wheel cache, and executions store absent from the executor.
Denial of service	Per-run resource caps (1 vCPU/512 MiB) and ephemeral teardown of every run	T3	No residual socket, ext4 disk, mount, or dataset after 9+ runs. <i>Not a primary focus (see Threats to Validity).</i>
Elevation of privilege	of Privilege separation across planes; dependency and code run in an isolated builder or in a networkless executor, T4 never in the control plane	T2, T2a, T2b, T4	A compromised dependency’s code executes but finds no secrets and no network route; build-time code starts inside the isolated builder rather than in the control plane.

Information disclosure is where the execution plane gives the strongest, most direct evidence. For network exfiltration (T1), the malicious application observes only the loopback and `sit0` pseudo-devices inside the guest: there is no `eth0`, because the execution microVM is launched without a network interface. All seven outbound attempts—DNS resolution, raw UDP and TCP to public resolvers, HTTPS, plain HTTP, and an SMTP probe—fail with `Network is unreachable` or name-resolution errors. The channel is closed by construction rather than by a firewall rule that an attacker might disable, which is the strongest form the property can take. For filesystem containment (T1b), the execution code runs as root inside the disposable guest but finds no control-plane secrets in its environment (`ASSET_ENCRYPTION_KEY`, `PRIVATE_KEY`, and related variables are all absent), and it can enumerate only its own input and output disks; sibling execution disks are not present. The files it can read (`/etc/passwd`, `/etc/shadow`) belong to the minimal ephemeral guest rootfs, not to the host, and contain no credential material. Finally, from a hypothetically compromised executor (T4), the asset-encryption key, the wallet private key, the IPFS repository, the wheel cache, and the executions store are all absent (Table 4); the same assets are colocated in the single namespace of Environment A. The static code surface mirrors this runtime containment: the execution plane that runs untrusted code is also the smallest first-party codebase—about 365 lines, against roughly 590 in the build plane and 1,500 in the secret-holding control plane—so the boundary an attacker reaches first is also the least code to audit and holds none of the control-plane logic.

Denial of service. Each run executes under fixed resource caps (1 vCPU/512 MiB) and is destroyed at teardown. The persistence test (T3) confirms this: after more than nine consecutive executions, the executor retains no Firecracker socket, no `rootfs-overlay`, `input`, `output`, or `application ext4` image, no residual loopback mount, and no leftover dataset or working directory. The `cleanup` trap removes the entire per-run working directory, so no ephemeral state survives to contaminate or starve a later run. We do not claim broader DoS resistance (e.g., against a flood of requests); resource scheduling and admission control are out of scope (see Threats to Validity).

Elevation of privilege. The supply-chain test (T2) connects the planes. We seeded a malicious package into the builder’s wheel cache and registered an application that declares it as a dependency. The builder resolved and bundled the package (a build that reported the cache hit), and the package then ran its payload at import time inside the execution microVM (T2a). That payload is the interesting evidence: the dependency’s code does execute, but it finds no control-plane secrets in the environment and reports its outbound probe to a public resolver as blocked, exactly mirroring the T1 and T1b results. A compromised dependency therefore inherits the containment of the plane that runs it. The complementary build-time vector (a source distribution whose `setup.py` runs during the build) begins executing inside the build microVM before failing under the read-only cache mount (T2b), which confirms that such code, when it does run, runs in the isolated builder rather than in the control plane.

6.3 Operational Cost

Table 2 consolidates the cost of the isolation strategy across architectures, and Table 3 reports the live A/B measurement in detail. End-to-end execution latency is indistinguishable between A and B (≈ 25 s each, measured on the same host). This time is dominated by the microVM life cycle—kernel boot, disk discovery, output-disk synchronization, and the guarded shutdown sequence—rather than by the application itself, whose computation over sixty rows completes in milliseconds.

Table 2. Comparative results across the isolation spectrum: a no-isolation monolith (M0), a centralized deployment that adds ephemeral microVMs but colocates all assets (A), the compartmentalized design that also separates the planes (B), and the SGX-based original DNAT as a cited reference. Execution latency and overhead are measured on the same host (2026-06-23); build sizes/latencies are from the live A/B campaign (2026-06-11); on-chain gas is measured on Hardhat; the SGX column (*cite*/—) is cited from the original DNAT work.

Metric	M0	A	B	SGX (ref.)
Execution latency, benign app (s)	0.07	25.2	25.2	<i>cite</i>
microVM boot/life-cycle overhead (s)	≈ 0	≈ 25.1	≈ 25.1	<i>cite</i>
Build latency, no deps (s)	— [‡]	30.2	30.1	—
Build latency, requests +4 (s)	— [‡]	34.9	38.2	—
Artifact size, no deps / requests (MiB)	—	16.0 / 19.6	16.0 / 19.6	—
On-chain gas, register / purchase (k) [§]	389 / 97	389 / 97	389 / 97	—
App reuse with deps, no recompile	yes	yes	yes	no

The microVM adds ≈ 25 s over M0’s ≈ 0.07 s, since the application’s compute over 60 rows is sub-second; that fixed cost is the price of a fresh, networkless microVM per run. $A \approx B$ by design, as both share the same Firecracker microVMs (live A/B cost in Table 3, blast radius in Table 4). [‡]M0 keeps the unchanged build microVM (only execution is direct), so its build cost equals A. On-chain gas is identical across M0/A/B because the contract is the shared control plane; an asset’s optional bloom filter adds ≈ 185 k gas per 256 B. [§]Gas is the EVM’s unit of work (a plain transfer is 21k); on a representative layer-2 (0.03 gwei, ETH at US\$3,000) the cost is $\approx$ US\$0.035 to register and $<$ US\$0.01 to purchase. The SGX column frames the reuse and dependency limitation of the original DNAT.

The no-isolation baseline (M0) makes this concrete: running the same workload directly in the container’s namespace—no Firecracker, no disk isolation, no network removal—the benign application completes in ≈ 0.07 s, so the microVM adds roughly 25 s of fixed per-execution overhead. That overhead is the price of launching a fresh, networkless microVM for every run, and it is exactly what buys the containment: in the same measurement, M0 exposes a working **eth0** and succeeds at network exfiltration, finds all six control-plane secrets in its environment, and persists its writes, whereas A and B (with the microVM) block

exfiltration, expose no secret, and leave no residual state. Separating the planes (B vs A) adds nothing on top of this microVM cost.

The on-chain cost is likewise architecture-independent. Registering an asset on the marketplace contract costs $\approx 389\text{k}$ gas (a dataset with an empty bloom filter; an application $\approx 327\text{k}$), and purchasing access costs $\approx 97\text{k}$ gas; a dataset’s optional on-chain bloom filter adds $\approx 185\text{k}$ gas per 256 bytes. These figures are identical for M0, A, and B because the registry is the shared control plane— isolation lives in the build and execution planes, not in the contract—so the blockchain cost of the user journey is unchanged by the isolation strategy. In concrete terms (a plain value transfer is 21k gas), on a representative layer-2 deployment at 0.03 gwei with ETH at US\$3,000 these amount to about US\$0.035 to register an asset and under US\$0.01 to purchase access, so the marketplace is inexpensive to operate where such applications realistically run.

The build plane is more expensive, as expected from its heavier configuration (two vCPUs, more memory, network egress, and dependency resolution). A dependency-free build completes in about 30 s and produces a 16 MiB artifact; a build that pulls a real dependency tree (`requests` and four transitive packages) takes 35–38 s and grows the artifact to 19.6 MiB. The two designs produce byte-identical artifacts because they share the same builder microVM. Crucially, separating the planes (B) adds no measurable cost over the centralized baseline (A) on either path: the cost is paid by the microVM, which both designs share.

6.4 Threats to Validity

The measurements were taken on a single host with a Linux-backed container runtime; absolute latencies will vary with hardware (a colder first campaign saw a 31.4 s execution mean and 103–131 s builds, with the same $A \approx B$ relationship), but the containment and exfiltration results are structural (absence of a network interface, absence of secrets) and do not depend on timing. The execution-plane tests bypass IPFS and the builder to isolate the microVM under study; the end-to-end path through the control plane is exercised separately by the access-control test. We report one residual weakness honestly: in the single-host deployment the three planes share one Docker network, so a compromised executor can still reach the control API and the blockchain RPC over that network, even though the secrets and stores that define the blast radius remain isolated; internal mutual TLS and network segmentation remain future work (Table 4). Finally, the DoS analysis covers only per-run state hygiene, not load-induced resource exhaustion, and the SGX column of Table 2 is cited from the original DNAT rather than re-measured here.

A more fundamental caveat concerns the execution substrate. The prototype runs each plane as a container with Firecracker microVMs rather than inside a hardware-backed confidential VM, so our measurements do not come from a genuinely confidential environment and we make no claim about confidential-computing overheads. This does not weaken the architectural argument: the properties we evaluate—privilege separation across planes, a networkless execution microVM, ephemeral teardown, and on-chain access control—are properties

of the architecture, not of the memory-encryption substrate. Running the same planes inside confidential VMs (e.g., AMD SEV-SNP [1]) would change the implementation and add a hardware-attestation and memory-encryption cost, but would preserve the containment and blast-radius results demonstrated here. The analysis therefore remains valid, with confidential execution as a substrate substitution rather than an architectural change.

6.5 A/B Comparison: Live Measurement of Both Environments

To remove any inference from the comparison, we ran the full suite against both deployments live and end to end: the compartmentalized system (Environment B, three containers `dnat-client`/`dnat-builder`/`dnat-executor`) and the centralized baseline (Environment A, a single `dnat-baseline` container that colocates the same logic). Because both environments share the same Firecracker execution and build microVMs, we expect—and observe—that the microVM-level properties and the operational cost are identical, while the architectural variable surfaces only in the blast radius.

The per-microVM guarantees are a constant. Network-exfiltration resistance (T1), filesystem containment (T1b), and persistence resistance (T3) produce byte-for-byte equivalent evidence in A and B: the execution guest exposes no usable network interface (all seven outbound channels fail at the syscall level with `ENOSYS/name resolution` errors), carries no control-plane secret in its environment, sees no sibling tenant disk, and—verified through the container’s read-write layer after the campaign’s runs—leaves behind no Firecracker socket, no per-run `ext4` disk, and no working directory; only the static, read-only rootfs template persists. Access control (T5) likewise holds in both: an execution request issued without a prior `purchaseAccess` is rejected with *access denied* by the on-chain `hasAccessByIds` check in A and in B alike, since that gate lives at the control plane, above the split.

Compartmentalization adds no measurable cost. Table 3 reports the operational cost on both paths. Across three applications per environment, end-to-end execution latency is indistinguishable between A and B (mean ≈ 26.8 s each), and a dependency-free build (30.1 vs 30.2 s) and a build that pulls `requests` with its transitive tree (38.2 vs 34.9 s) match within run-to-run variance, producing byte-identical artifacts (16.0 and 19.6 MiB). The cost is paid by the microVM life cycle, which both designs share; separating the planes does not add to it.

Plane separation changes only the blast radius. The single place where A and B diverge is cross-plane reach (Table 4, now populated from live inspection rather than configuration). In B, the executor and the builder carry *none* of the control-plane secrets (`ASSET_ENCRYPTION_KEY`, `PRIVATE_KEY`, the service URLs) in their environment, and the sensitive stores (IPFS repository, wheel cache, executions store) are absent from their namespace; the secrets live only in the control plane, where they belong. In A, the very same secrets and stores sit in the single namespace that also runs the execution logic. The supply-chain test (T2) inherits this split: a compromised dependency’s code executes, but in B

it executes inside a plane (build or execution) that holds neither secrets nor a route to them, whereas in A the same code lands beside every sensitive asset. This is the paper’s comparative thesis—*containment, not absolute security*—now demonstrated with both environments measured, not one measured and one argued.

Table 3. Operational cost of the ephemeral-microVM isolation, measured live in both environments. Compartmentalization (B) adds no execution or build overhead over the centralized baseline (A); both share the same Firecracker microVMs.

Metric	Env. B	Env. A
Execution latency — <code>t1_network_exfil</code>	27.3 s	27.4 s
Execution latency — <code>t1b_fs_escape</code>	26.6 s	26.5 s
Execution latency — <code>benign_stats</code>	26.5 s	26.6 s
Execution latency — mean ($n=3$)	26.8 s	26.8 s
Build latency — no dependencies	30.1 s	30.2 s
Build latency — <code>requests</code> (+4 transitive)	38.2 s	34.9 s
Artifact <code>application.ext4</code> — no deps	16.0 MiB (identical)	
Artifact <code>application.ext4</code> — <code>requests</code>	19.6 MiB (identical)	

Wall-clock from the test host (warm), measured 2026-06-11. The $A \approx B$ equality is the structural result; absolute values track host warmth (a colder first campaign saw a 31.4 s execution mean and 103–131 s builds, with the same $A \approx B$ relationship).

Table 4. Reach of a compromised execution plane over control-plane assets: compartmentalized DNAT (B) versus centralized baseline (A), from live inspection. “Isolated” = absent from the plane’s namespace; “Colocated” = same namespace as the execution logic.

Control-plane asset from the execution plane	reachable Environment B (DNAT)	Environment A (baseline)
Asset-encryption key (<code>ASSET_ENCRYPTION_KEY</code> , env)	Isolated (NOT SET)	Colocated (SET)
Wallet private key (<code>PRIVATE_KEY</code> , env)	Isolated (NOT SET)	Colocated (SET)
Service URLs (<code>RPC_URL</code> , <code>IPFS_API_URL</code> , <code>BUILDER_URL</code> , <code>EXECUTOR_URL</code> , env)	Isolated (NOT SET)	Colocated (SET)
IPFS repository, wheel cache, executions store (filesystem)	Isolated (separate namespace)	Colocated (same filesystem)
IPFS API (port 5001, network)	Blocked (loopback in CVM1)	Reachable (127.0.0.1)
Control API (port 3001, network)	Reachable [†]	Reachable (127.0.0.1)
Blockchain RPC (port 8545, network)	Reachable [†]	Reachable (127.0.0.1)

[†] Residual risk in the single-host deployment (the three containers share one Docker network, `dnat-runtime`). In a distributed deployment across separate machines this is governed by inter-host network policy. This substantiates the threat model’s stated limitation: internal mutual TLS and network segmentation remain future work. The defining assets of the blast radius (secrets and persistent stores) are nonetheless isolated in B and colocated in A.

7 Related Work

8 Concluding Remarks

9 Conclusion

This paper framed the DNAT implementation as a layered containment architecture for a confidential data marketplace, and then tested that framing against the running prototype. The central design move—separating control, build, and execution into planes with different network and persistence properties—was evaluated not as an abstract claim of trustlessness but as a measurable reduction in blast radius relative to a centralized baseline that differs only in the colocation of assets.

The experimental results support the comparative argument. Execution-time code cannot open outbound channels because the execution microVM is launched without a network interface; every probed channel failed with a structural network-unreachable error rather than a defeatable policy. Ephemeral state does not survive: after repeated executions the executor retained no disks, mounts, sockets, or datasets. The assets that define the blast radius—the asset-encryption key, the wallet key, the IPFS repository, the wheel cache, and the executions store—are absent from the compromised executor in the compartmentalized design, whereas they are colocated in the baseline. Execution is gated on chain, and the isolation is bought at a fixed operational cost of roughly thirty seconds of microVM life cycle per run.

At the same time, the evaluation makes the limitations concrete rather than rhetorical. In a single-host deployment the planes share one network, so a compromised executor can still reach the control API and the blockchain RPC even though the secrets and stores remain isolated; this directly motivates the pending work on mutual TLS and internal network segmentation. Dataset confidentiality still depends on preparation and orchestration outside the encrypted-artifact path, and the system does not yet expose a remote-attestation flow that would let an external party verify the software identity of each confidential component before trusting a result. The remaining empirical gap is a full live intrusion comparison against the centralized baseline; the present blast-radius comparison is derived from the deployment configuration, since both environments share the same execution sandbox and differ only in colocation.

We therefore conclude that the compartmentalized design is justified as a credible high-assurance baseline for a confidential marketplace prototype: it demonstrably narrows what a single compromise can reach and what it can leave behind, while leaving a clearly bounded set of properties—service identity, dataset confidentiality, and attestation—as the next steps. Future work will close the network-segmentation gap between planes, extend artifact-grade encryption and provenance to datasets and cached wheels, and add attestation so that the containment demonstrated here can be verified remotely rather than assumed.

Declaration on the Use of Generative AI

During the preparation of this work, the authors used generative AI tools (large language models) to assist with drafting and editing parts of the manuscript text and with the development and debugging of the prototype implementation and evaluation code. After using these tools, the authors reviewed, verified, and edited all resulting content as needed and take full responsibility for the entire content of this paper. No AI tool is listed as an author, and no AI tool was used to generate or fabricate experimental data, results, or scientific claims.

References

1. Advanced Micro Devices, Inc.: AMD SEV-SNP: Strengthening VM isolation with integrity protection and more. Tech. rep., Advanced Micro Devices, Inc. (2020)
2. Agache, A., Brooker, M., Iordache, A., Liguori, A., Neugebauer, R., Piwonka, P., Popa, D.M.: Firecracker: Lightweight virtualization for serverless applications. In: 17th USENIX Symposium on Networked Systems Design and Implementation (NSDI '20). pp. 419–434. USENIX Association (2020)
3. Benet, J.: IPFS - content addressed, versioned, P2P file system. CoRR **abs/1407.3561** (2014), <https://arxiv.org/abs/1407.3561>
4. Buterin, V.: A next-generation smart contract and decentralized application platform. Ethereum White Paper (2014), <https://ethereum.org/en/whitepublication/>
5. Jr, J.R.N., Nunes, J.B.S., Falcão, E.L., Sampaio, L., Brito, A.: On the tracking of sensitive data and confidential executions. In: Proceedings of the 14th ACM International Conference on Distributed and Event-based Systems (DEBS '20). pp. 1–10. ACM, Virtual Event, QC, Canada (2020). <https://doi.org/10.1145/3401025.3404097>
6. Nakamoto, S.: Bitcoin: A peer-to-peer electronic cash system. White Paper (2008), <https://bitcoin.org/bitcoin.pdf>
7. Shostack, A.: Threat Modeling: Designing for Security. John Wiley & Sons, Indianapolis, IN (2014)